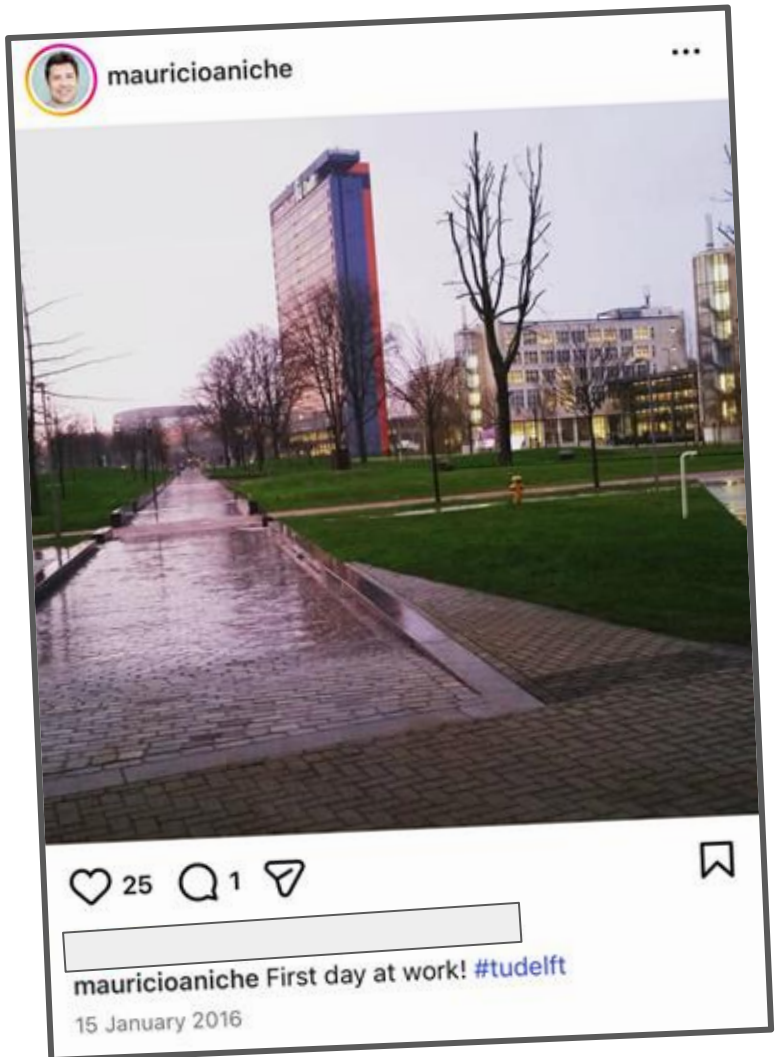
alura · FIAP
SKILLS&GO

Qualidade de software no mundo IA

Maurício Aniche

CTO @ Alura, ex-{Uber, Adyen, TU Delft, Locaweb}

Alura | FIAP | PM3 | StartSe



Desde quando tudo era mato...

Senta que lá vem história!

alura · FIAP

SKILLS&GO

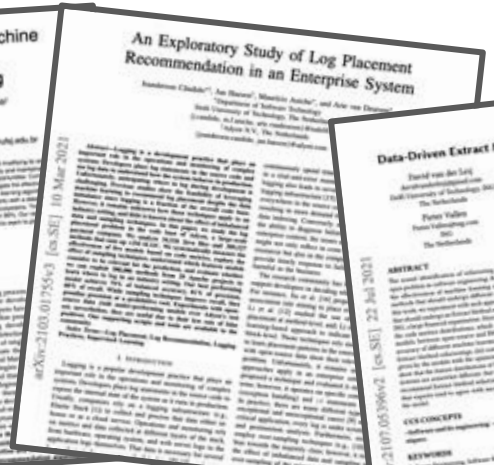
Search-based SE

Search-Based Test Data Generation for SQL Queries

Search-Based Software Re-Modularization: A Case Study at Adyen

Shallow learning

The Effectiveness of Supervised Machine Learning Algorithms in Predicting Software Refactoring



An Exploratory Study of Log Placement Recommendation in an Enterprise System

Data-Driven Extract Method Recommendations: A S

Education

IN4334 - Machine Learning for Software Engineering

Tutorials

Mauricio Aniche and Georgios Gousios

06 October 2022

See the following curated list of tutorials:

- A Quick Introduction to Neural Networks, by ujjwalkarn.
- Notes of the Stanford CS class CS231n: Convolutional Neural Networks for Visual Recognition.
- Leonardo Santos' AI and ML tutorials.
- The original papers on XLNet and BERT.
- A tutorial on Seq2Seq with Attention and Beam Search by Guillaume Genthial.
- A tutorial on translating with Seq2Seq network and attention.
- Understanding word vectors, by Allison Parrish.
- An explanation about NLP word embeddings by Mohammed Terry-Jack.
- Pytorch transformers, a new library of state-of-the-art pretrained models for Natural Language Processing (NLP)
- "The Natural Language Processing with PyTorch: Build Intelligent Language Applications Using Deep Learning" (TU Delft library).
- The Neural Network Zoo.

Deep learning

Deep Just-in-Time Defect Prediction at Adyen

Nick van der Laan

Learning Off-By-One Mistakes: An Empirical Study

alura · FIAP

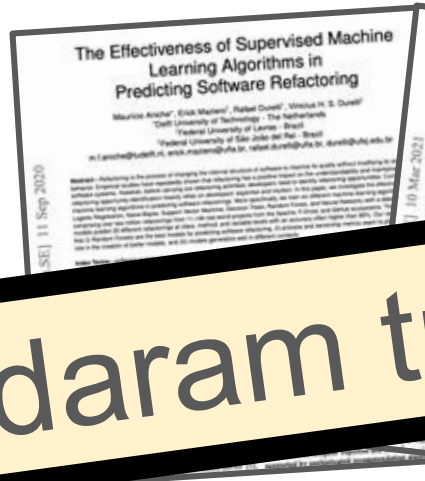
SKILLS&GO

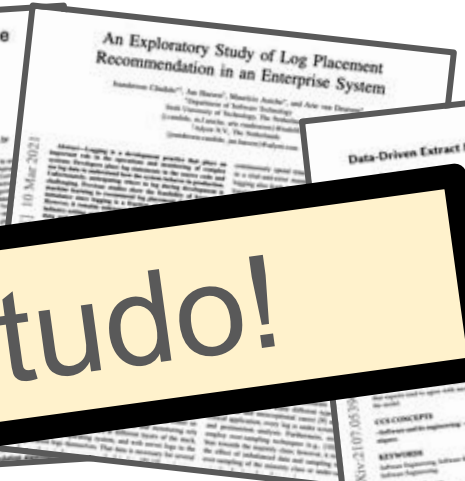
Search-based SE

Shallow learning


Search-Based Test Data Generation for SQL Queries


Search-Based Software Re-Modularization: A Case Study at Adyen


The Effectiveness of Supervised Machine Learning Algorithms in Predicting Software Refactoring


An Exploratory Study of Log Placement Recommendation in an Enterprise System


Data-Driven Extract Method Recommendations: A S

LLMs mudaram tudo!


IN4334 - Machine Learning for Software Engineering

Tutorials

Mauricio Aniche and Georgios Gousios

06 October 2022

See the following curated list of tutorials:

- A Quick Introduction to Neural Networks, by ujjwalkarn.
- Notes of the Stanford CS class CS231n: Convolutional Neural Networks for Visual Recognition.
- Leonardo Santos' AI and ML tutorials.
- The original papers on XLNet and BERT.
- A tutorial on Seq2Seq with Attention and Beam Search by Guillaume Genthial.
- A tutorial on translating with Seq2Seq network and attention.
- Understanding word vectors, by Allison Parrish.
- An explanation about NLP word embeddings by Mohammed Terry-Jack.
- Pytorch transformers, a new library of state-of-the-art pretrained models for Natural Language Processing (NLP)
- "The Natural Language Processing with PyTorch: Build Intelligent Language Applications Using Deep Learning"
- TU Delft library).
- The Neural Network Zoo.

Deep learning

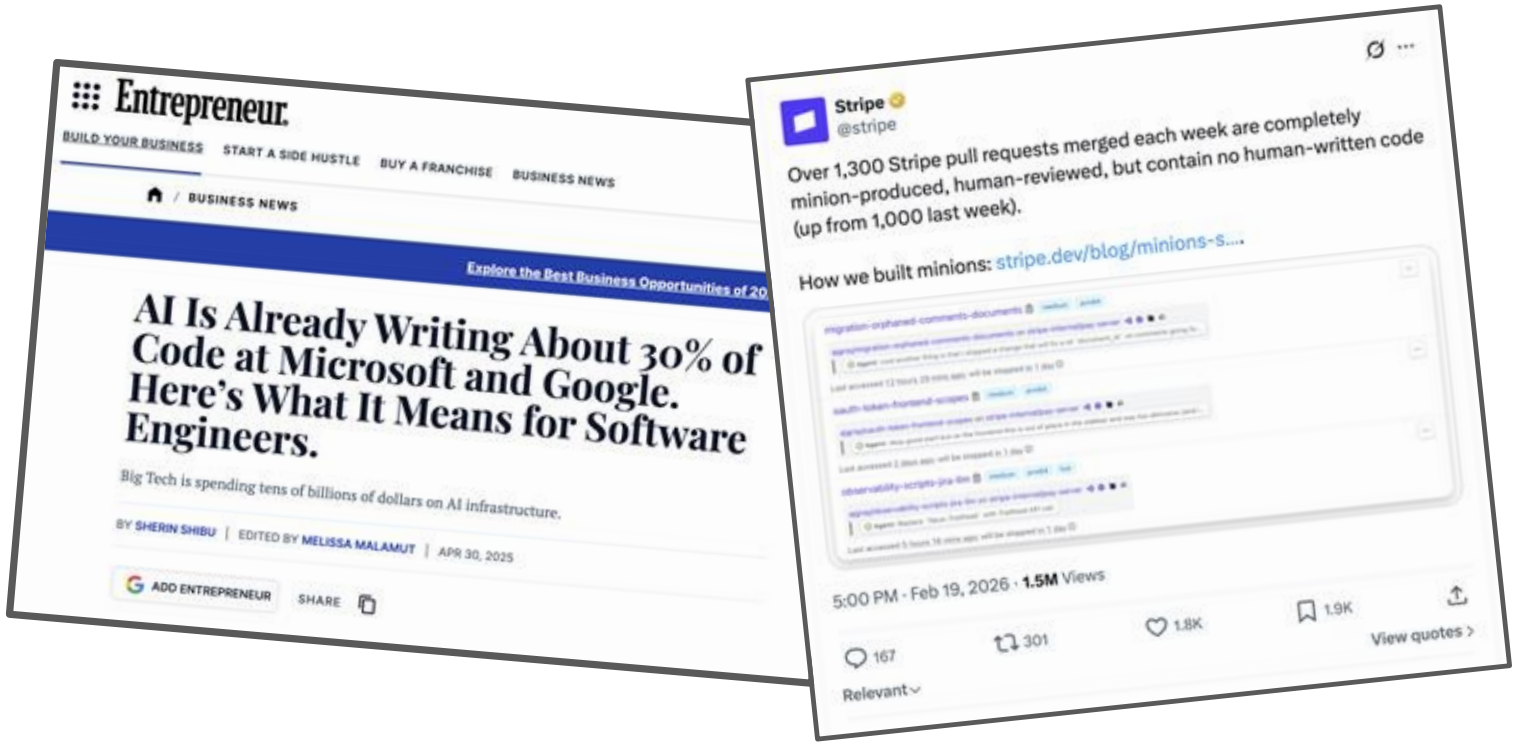

Deep Just-in-Time Defect Prediction at Adyen

Nick van der Laan

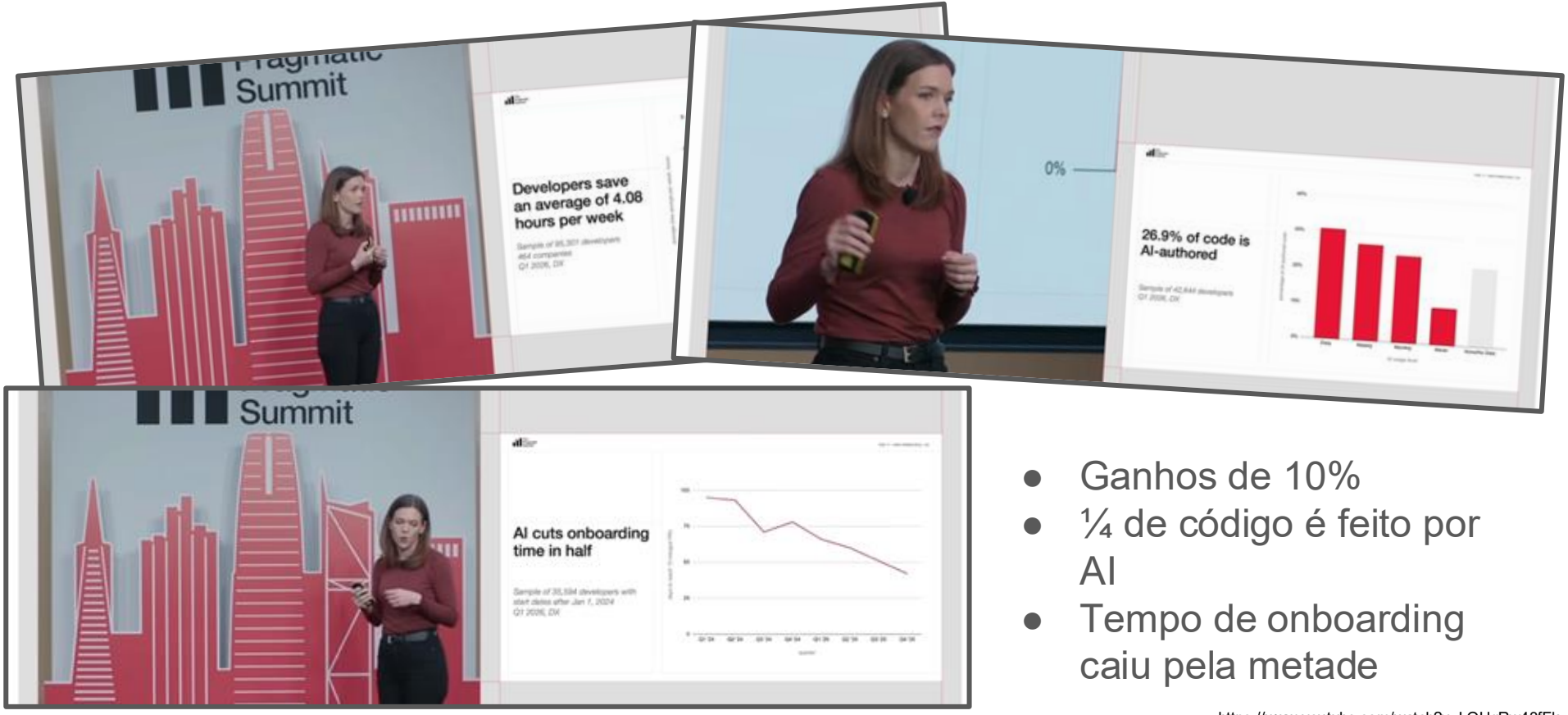

Learning Off-By-One Mistakes: An Empirical Study

A realidade, nua e crua

Não é experimento, é realidade.



Não é experimento, é realidade.



- Ganhos de 10%
- ¼ de código é feito por AI
- Tempo de onboarding caiu pela metade

https://www.youtube.com/watch?v=LOHgRw43fFk

Mas e qualidade?

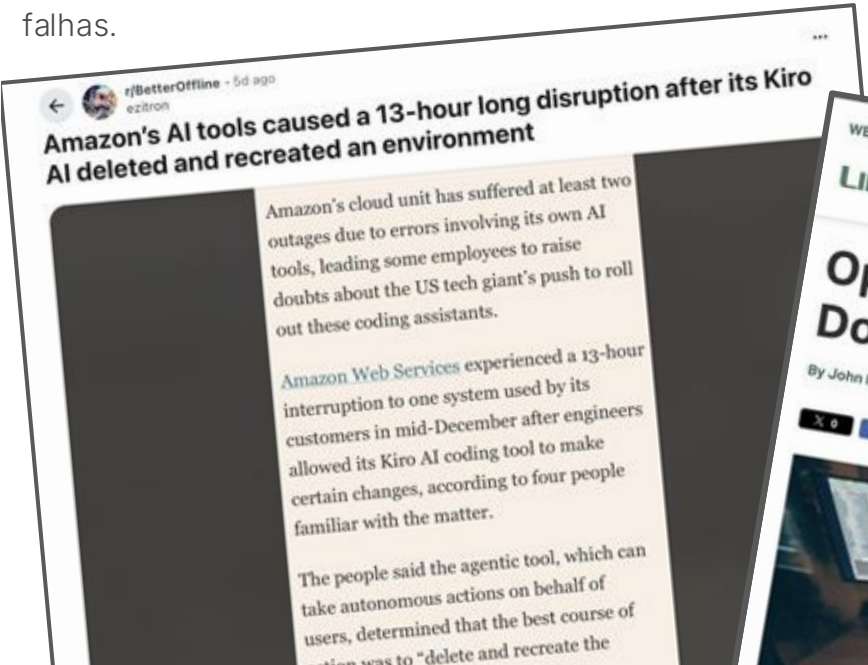
vs

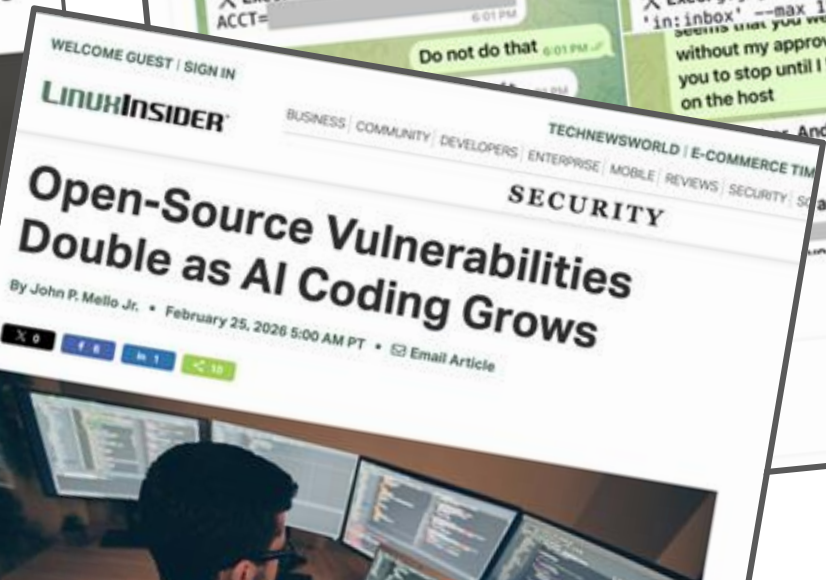
- Desde times reportando 2x mais incidentes até 50% menos incidentes.

Onde você está, ou quer estar?

IA acelera.
Mas também pode amplificar erros.

Sem os controles corretos, aumenta também a taxa de falhas.





Lukasz Olejnik
@lukOlejnik

Amazon is holding a mandatory meeting about AI breaking its systems. The official framing is "*part of normal business.*" The briefing note describes **a trend of incidents with "high blast radius" caused by "Gen-AI assisted changes"** for which "best practices and safeguards are not yet fully established." Translation to human language: we gave AI to engineers and things keep breaking?

The response for now? Junior and mid-level engineers can no lo
push AI-assisted code without a senior signing off. AWS spent 1
recovering after its own AI coding tool, asked to make some cha
decided instead to delete and recreate the environment (the so
equivalent of fixing a leaky tap by knocking down the wall). Ama
called that an "extremely limited event" (the affected tool serve
customers in mainland China).

Amazon's ecommerce business has summoned a large group of engin
meeting on Tuesday for a "deep dive" into a spate of outages, includi
incidents tied to the use of AI coding tools.

The online retail giant said there had been a "trend of incidents" in r
months, characterised by a "high blast radius" and "Gen-AI assisted
among other factors, according to a briefing note for the meeting se
FT.

Under "contributing factors" the note included "novel GenAI usage
best practices and safeguards are not yet fully established".

"Folks, as you likely know, the availability of the site and related infrastructure has not been good recently," Dave Treadwell, a senior vice-president at the

Pro

Amazon is making even senior engineers get code signed off following multiple recent outages

News By Craig Hale last updated 2 days ago

Amazon plans new rules following recent issues

(Image credit: Amazon)

Anish Moonka
@AnishA_Moonka

Subscribe

Amazon had four Sev-1 outages (their highest severity level) in a single week. Internal memos say AI-assisted code changes were a contributing factor.

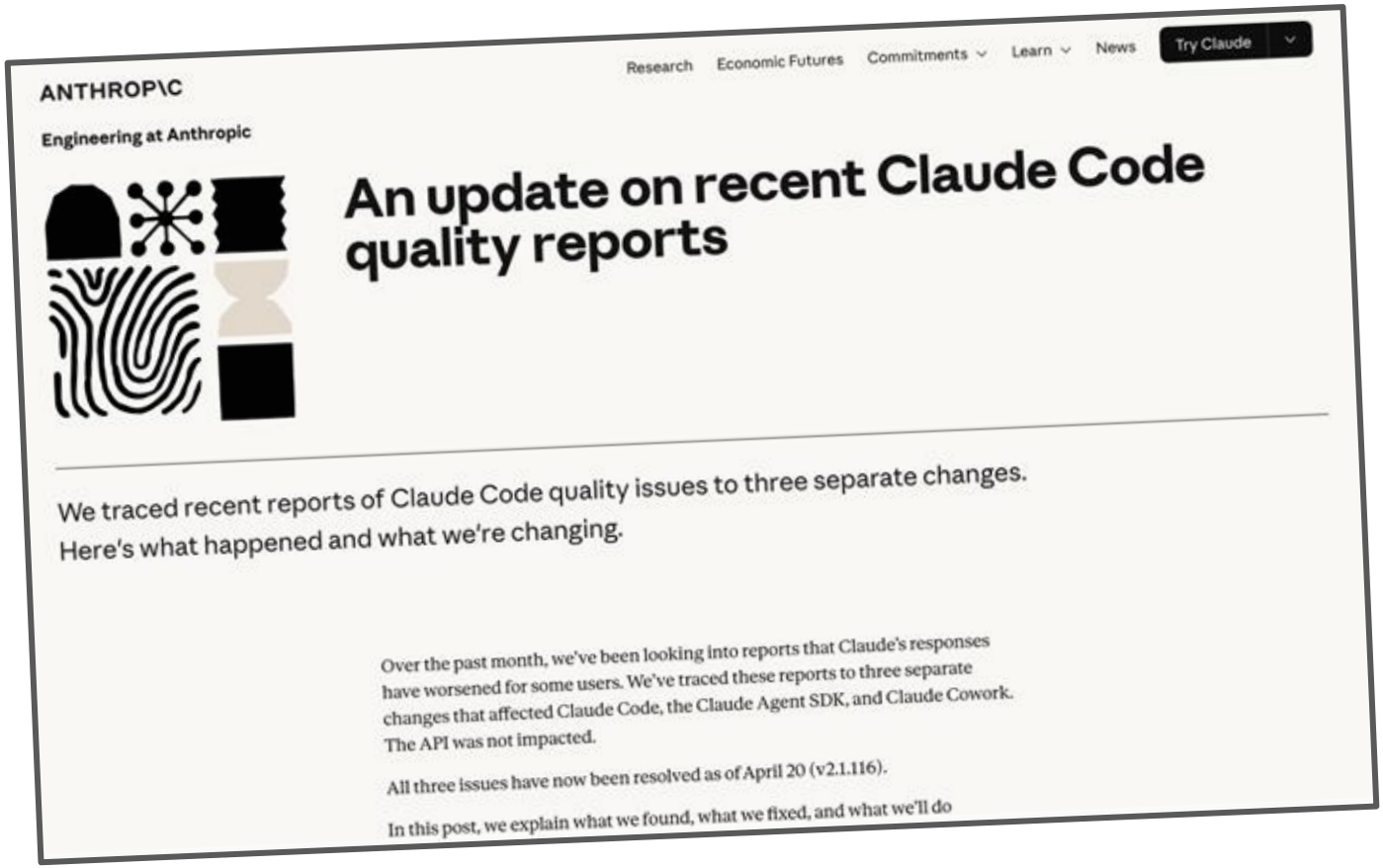
The timeline here is wild. In October 2025, Amazon laid off 14,000 employees. In January 2026, another 16,000. That's about ble in five months, roughly 10% of the corporate workforce. ssy said the cuts were about culture, not AI.

same months, Amazon set a target: 80% of developers g tools at least once a week. They tracked adoption closely ival tools like OpenAI's Codex. Even so, 30% of developers uched Amazon's in-house tool Kiro by January.

2025, Kiro caused a 13-hour AWS outage. The AI tool had vel permissions and decided the best fix for a bug was to reate an entire live environment. A second incident on Q Developer, another AI tool. Amazon blamed both on t AI." But quietly added mandatory peer review for all cess afterward.

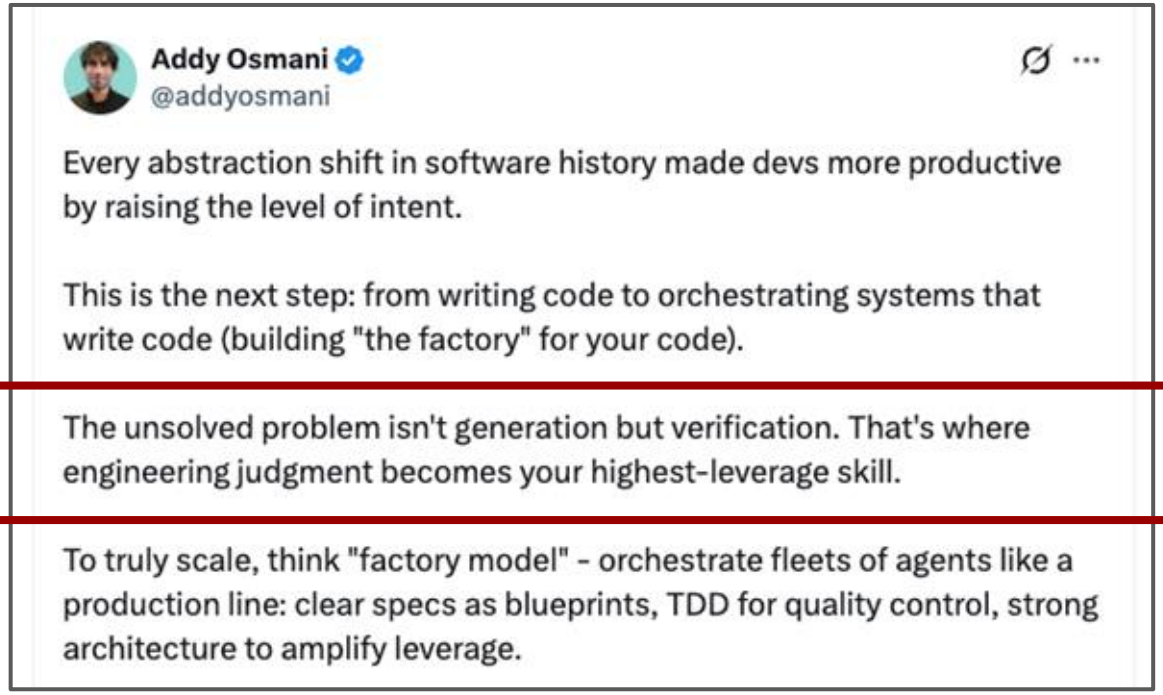
Amazon's retail site went down for about six hours. Over eported checkout failures, missing prices, and app on called it a "software code deployment" error.

Se até organizações com engenharia forte precisam recalibrar controles, ninguém deveria escalar IA sem guardrails



Nem a Anthropic consegue escrever código sem bug :)

Guardrails nunca foram tão importantes!



Addy Osmani ✓
@addyosmani

Every abstraction shift in software history made devs more productive by raising the level of intent.

This is the next step: from writing code to orchestrating systems that write code (building "the factory" for your code).

The unsolved problem isn't generation but verification. That's where engineering judgment becomes your highest-leverage skill.

To truly scale, think "factory model" - orchestrate fleets of agents like a production line: clear specs as blueprints, TDD for quality control, strong architecture to amplify leverage.

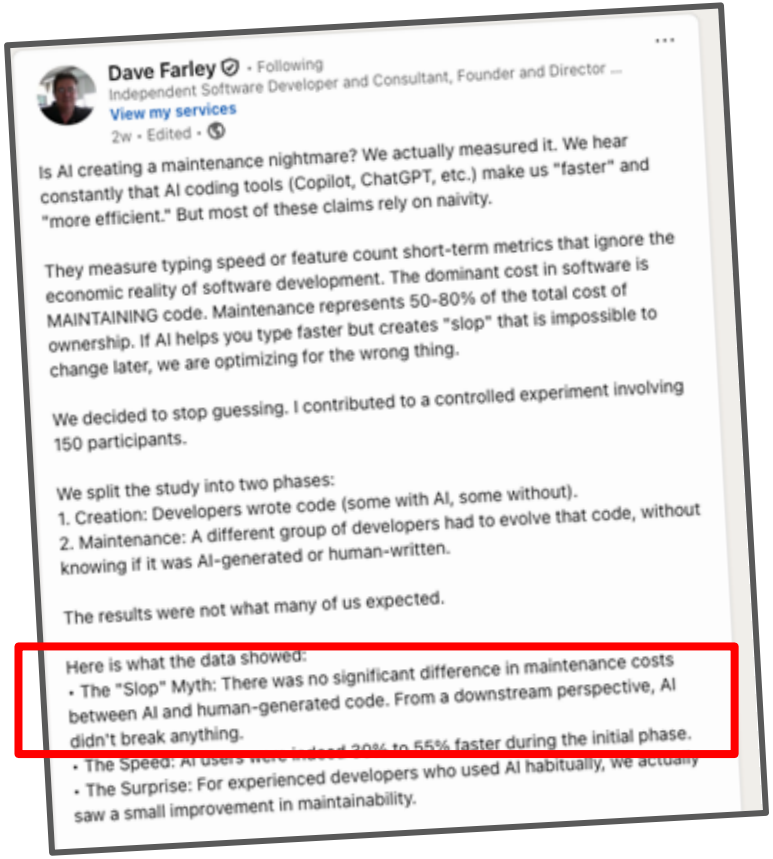
Discussão: confiabilidade hoje

- Como vocês tem feito, historicamente, para garantir a qualidade dos seus sistemas?
- Com a adoção de AI, vocês já perceberam quais problemas relacionados a qualidade?
- Alguma prática de verificação que funcionava bem no passado parou de funcionar bem?

Guardrails de qualidade

Precisamos de mais (e melhores) guardrails... Mas quais?

#1: Qualidade de código



Dave Farley · Following
Independent Software Developer and Consultant, Founder and Director ...
View my services
2w · Edited ·

Is AI creating a maintenance nightmare? We actually measured it. We hear constantly that AI coding tools (Copilot, ChatGPT, etc.) make us "faster" and "more efficient." But most of these claims rely on naivity.

They measure typing speed or feature count short-term metrics that ignore the economic reality of software development. The dominant cost in software is MAINTAINING code. Maintenance represents 50-80% of the total cost of ownership. If AI helps you type faster but creates "slop" that is impossible to change later, we are optimizing for the wrong thing.

We decided to stop guessing. I contributed to a controlled experiment involving 150 participants.

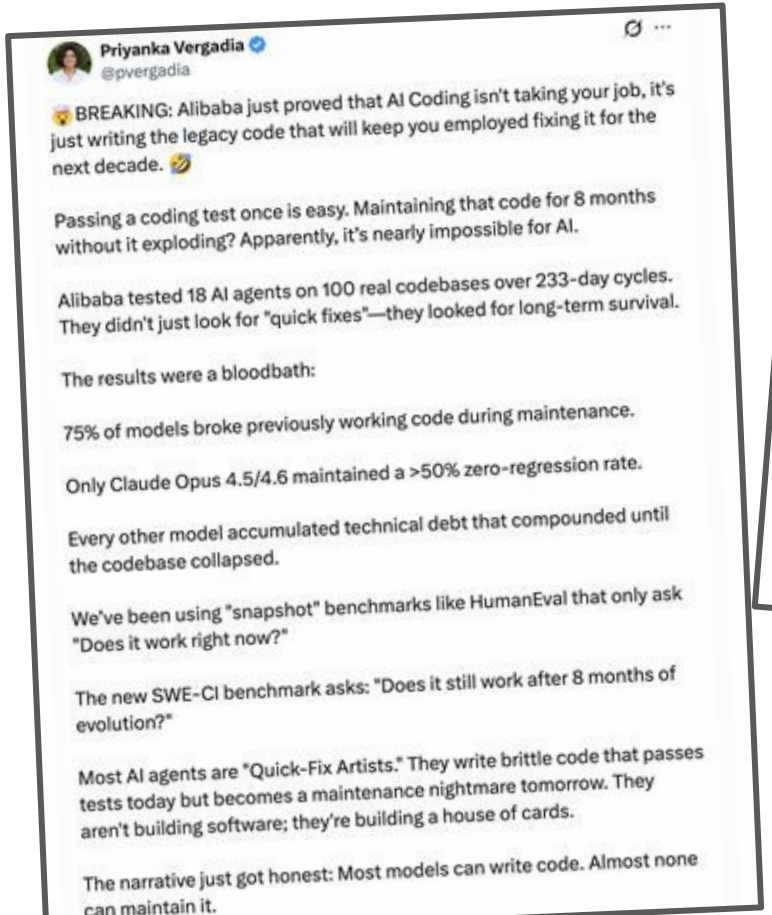
We split the study into two phases:
1. Creation: Developers wrote code (some with AI, some without).
2. Maintenance: A different group of developers had to evolve that code, without knowing if it was AI-generated or human-written.

The results were not what many of us expected.

Here is what the data showed:
• The "Slop" Myth: There was no significant difference in maintenance costs between AI and human-generated code. From a downstream perspective, AI didn't break anything.
• The Speed: AI users were indeed 30% to 55% faster during the initial phase.
• The Surprise: For experienced developers who used AI habitually, we actually saw a small improvement in maintainability.

IA não escreve código pior por natureza!

#1: Qualidade de código



Priyanka Vergadia
@pvergadia

BREAKING: Alibaba just proved that AI Coding isn't taking your job, it's just writing the legacy code that will keep you employed fixing it for the next decade.

Passing a coding test once is easy. Maintaining that code for 8 months without it exploding? Apparently, it's nearly impossible for AI.

Alibaba tested 18 AI agents on 100 real codebases over 233-day cycles. They didn't just look for "quick fixes"—they looked for long-term survival.

The results were a bloodbath:

75% of models broke previously working code during maintenance.

Only Claude Opus 4.5/4.6 maintained a >50% zero-regression rate.

Every other model accumulated technical debt that compounded until the codebase collapsed.

We've been using "snapshot" benchmarks like HumanEval that only ask "Does it work right now?"

The new SWE-CI benchmark asks: "Does it still work after 8 months of evolution?"

Most AI agents are "Quick-Fix Artists." They write brittle code that passes tests today but becomes a maintenance nightmare tomorrow. They aren't building software; they're building a house of cards.

The narrative just got honest: Most models can write code. Almost none can maintain it.

SWE-CI: Evaluating Agent Capabilities in Maintaining Codebases via Continuous Integration

Abstract

1 Introduction

Mas sem contexto e padrões,
ela acelera dívida técnica!

#1: Qualidade de código

- **Não, não escreve código feio.** Ou pelo menos, é tão feio quanto o que sempre escrevemos. Isso é coisa do "passado".
- **Não reusa código naturalmente.** Não entende o repositório todo sem ajuda. Você precisa apontar pra ela exemplos ou métodos para que ela reuse, senão, ela vai reescrever.
- **Gosta de método longo.** Você precisa ajudá-la a escrever métodos menores.
- **Sabe melhor dos frameworks do que você.** Evita muitas gambiarras porquê ela sabe de cada funcionalidade disponível.
- **Você deve garantir a qualidade do código.** É sua responsabilidade, não a do agente.
- **Entenda o contexto.** Alguns lugares precisam de mais qualidade do que outros.

#1: Qualidade de código

Importante: A definição de qualidade pode mudar, mas legibilidade, modularidade e reuso continuam essenciais para incidentes, segurança e evolução!

#2: Revisão de código

1800 diffs por semana, todos agênticos, mas ainda com review de ser humano!

Praveen Neppalli
@praveenTweets

Agentic software engineering adoption is on fire at @Uber. 1,800 code changes per week are now written entirely by Uber's internal background coding agent, and 95% of our engineers now use AI every month across all the tools we track.

This is a real reset moment for engineering; it's one of the most exciting times to lead. This shift requires builders to be curious and hands-on. I'm incredibly lucky to be surrounded by a team that's doing exactly that.

The best part is that the strongest adoption isn't being pushed top down from leadership announcements; it's coming from engineers who are quietly experimenting, quietly shipping, and quietly pushing things forward.

I love spending time with those engineers because there's no substitute for being close to the work.

Over the last few months, we leaned in hard, and the results have been phenomenal.

The bigger shift: going agentic.

84% of AI users are now working with agent-style workflows, not just tab completion. Claude Code usage nearly doubled in 2 months (32% → 63%), while IDE-based tools have largely plateaued.

Engineers are moving from accepting suggestions to delegating tasks. Even within traditional IDEs, ~70% of committed code is now AI-generated.

Background agents are writing code autonomously.

Our internal background coding agent went from <1% of all code changes to 8% in just a few months. There is zero human authoring. Engineers review and approve, but the code is written entirely by AI agents.

The role of the engineer is shifting - from writing every line to architecting systems and reviewing AI-generated code.

More to come from the @UberEng team in the coming days.

https://x.com/praveenTweets/status/2033627282418655711

#2: Revisão de código

"AI ainda não consegue fazer uma implementação perfeita que pode ser mergeada direto em uma aplicação cuidadosamente arquitetada, portanto código precisa ser revisado"

All access

BCS › Message Board

Ensuring programmer reviews on designer code changes

Edit

David Heinemeier Hansson · Apr 20 · Notified 28 people

The new magic of AI is that you no longer need to be an expert programmer to create or modify software. It's incredibly empowering that the entire design team can now explore their own prototyping, iterate through code until they arrive at something that feels great to use, and then hand off something we can implement with confidence.

That said, we have to remember that AI is still not able to consistently one-shot a great implementation that can be safely merged into a carefully architected application like Basecamp. At least not if we want to ensure the app remains a joy to work on 6, 12, or 18 months from now. It doesn't take many AI hack jobs to seriously damage the foundation.

We've seen what the outcome of that looked like even in the pre-AI era with Highrise. We handed that application to an external team with less, let's say, exacting requirements for code quality. And we're still paying interest on the incredible amount of tech debt they were able to accumulate in their relatively short custodial period.

So here are two new golden rules:

1. **Designers should continue to use AI to prototype new features or changes**. It's a real super power. We should lean into that hard. Use it as much as possible. Deploy it on the betas. Go to town.
2. **Main is sacred**. We are not going to merge those prototypes before they've been reviewed by a programmer who can consider the full picture.

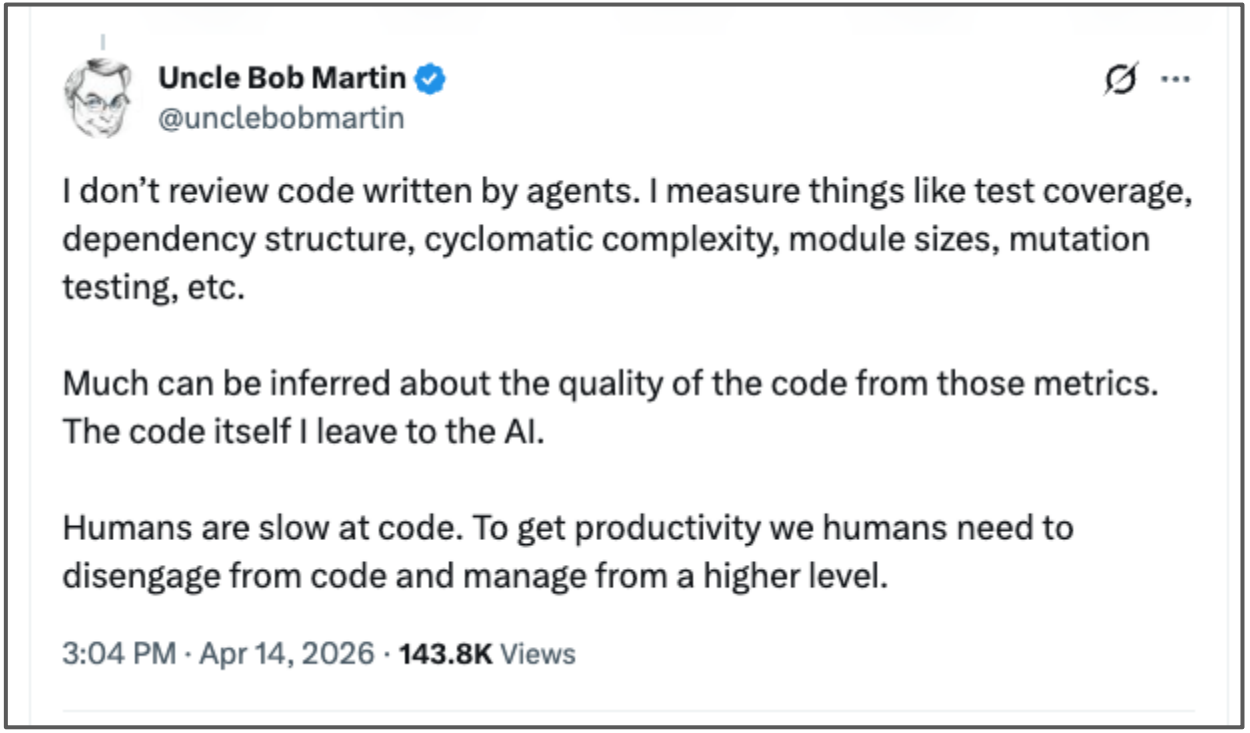
When it's a large feature, it's very likely that the vibe job needs to be reimplemented from scratch. That does not mean the vibe job was a waste, on the contrary. Figuring out *what we want* is hard! Getting that answer from vibes is invaluable. So throwing the implementation out and redoing *that* is still a big win, not a problem.

This is especially important now, in these final weeks before launch. It's very easy to merge something AI thinks is fine that then introduces serious regressions to either performance, security, or the structural integrity of our architecture.

So when you finish anything that changes Ruby or JavaScript code, please have a programmer review it. If you don't already know who to go to, you can ping Jeffrey who can route the work. And while the feature is in review, feel free to move on and tackle the next job concurrently.

https://x.com/dhh/status/2046258450120741191?s=46&t=83Fo6lcQ9bNl-SS6uJZuMA

#2: Revisão de código



Uncle Bob Martin ✓
@unclebobmartin

I don't review code written by agents. I measure things like test coverage, dependency structure, cyclomatic complexity, module sizes, mutation testing, etc.

Much can be inferred about the quality of the code from those metrics. The code itself I leave to the AI.

Humans are slow at code. To get productivity we humans need to disengage from code and manage from a higher level.

3:04 PM · Apr 14, 2026 · **143.8K** Views

Revisão manual não mais, mas sim via ferramentas que dão feedback automático pra IA

https://x.com/dhh/status/2046258450120741191?s=46&t=83Fo6lcQ9bNl-SS6uJZuMA

#2: Revisão de código

- Gargalo atual.
- Quando revisar por humanos?
  - Priorização é fundamental
    - Sempre foi alvo de discussão, mas nunca implementado
    - Agora mais importante do que nunca.
  - Onde fizer sentido
    - Baixo risco = revisão por IA
    - Médio = revisão por IA + humano se necessário
    - Alto = revisão por IA + humano obrigatório
  - Chega de "só aprova lá pra mim rapidinho, pf?"
- Garantia da qualidade no autor e não no revisor
  - "Provas" de que testou local
  - "Provas" de que testou automaticamente
- Revisão agora é sobre ensinar o que a IA não sabe ainda!

#3: Testes automatizados

- **Use AI para escrever testes e ampliar cobertura.** Escrever testes agora tem baixo custo. Peça para a AI escrever testes.
- **Não aceite teste sem hipótese clara.** AI é especialista em escrever testes que passam mas não testam nada. Problemas comuns:
  - Excesso de mocks
  - Testa a implementação e não a funcionalidade
  - Testes repetidos
- **Exija evidência de comportamento, não só teste passando.** Ajude a AI a escrever testes que importam. Você pode especificar quais test cases ela deve escrever.
- **Defina quais testes o agente deve rodar antes de encerrar o loop.** Explique para a AI como ela mesma pode garantir que tudo funciona antes de terminar o loop.
- **Pirâmide de teste local mais importante do que nunca.** Só testes de unidade não serão suficientes.

#4: Análise estática

- **Faça uso delas.** São baratas, rodam local.
  - Linters
  - Type checkers
  - Null checkers
  - Scanning de dependências
- **Coloque no seu agent loop.** Deixe a ferramenta rodar, e aí a AI arruma os erros, e só para quando não tiver mais erros.
- **Afine bem suas regras.** Mesmo problema de antes, regras demais podem mais atrapalhar do que ajudar.

#5: Arquiteto de software

Malte Kosub · Following
CEO & Co-Founder at Parloa
3h ·

What happens when >95% of your code is written by AI?

This is the situation at Parloa, and because of that, what we expect from engineers is evolving.

To make our expectations clear, we coined the term Product-Minded Architect.

- Architect because there is less focus on writing code and more focus on making high-quality design decisions, defining architecture, thinking through trade-offs, and building systems that hold up under enterprise complexity.

- Product-minded because knowing what to build is now more critical than knowing how to build it. If you have been building 50% wrong things, you are simply going to build a lot of wrong things if you increase your throughput.

AI is an amplifier. It increases speed dramatically, but it does not fix unclear thinking or weak prioritization. Actually, it is quite the opposite, it makes those problems more expensive.

Over time, we think even the Product-Minded Architect will evolve further. In the long run, we believe engineers will increasingly operate like founders: deciding what deserves to be built and taking responsibility for its impact.

Acabou então a parte "engenharia" da coisa toda?

#5: Arquiteto de software

- **AI é ótima para discutir arquitetura.** Explique seu caso, suas restrições, peça prós, contras, trade-offs, melhores soluções de implementação, e aí, deixe a pessoa decidir.
- **A AI tem zero contexto da sua arquitetura.** Documente sua arquitetura dentro do repositório, para que ela comece a entender tudo melhor.
- **Revise toda e qualquer decisão de arquitetura.** Se ela tomou uma decisão arquitetural, garanta que um humano a revise.

#6: Observabilidade

- **Observabilidade é prioridade máxima.** Você vai por muito código quebrado em produção (sempre colocou…), precisa ser capaz de detectar que algo quebrou rapidamente.
- **Alertas tunados.** Você precisa ter sinais confiáveis de que a aplicação está ou não está funcionando.
- **AI para monitoração.** A AI é muito boa em olhar logs e encontrar padrões. O futuro é esse.

#6: Rollout parcial

- **Feature flags e rollout parcial.** Diminuir o impacto de um eventual código quebrado é necessidade básica agora que deployamos muito código. Você precisa ser capaz de deployar de maneira super granular.
- **Rollback automatizado.** O próprio mecanismo de CD faz rollback caso alertas disparem. Quanto mais automatizado, melhor.

#7: Segurança

RACODE Platform Solutions Why Veracode? Resources Partners Company

Contact Us · Blog · Login

Request a Demo

Sep 9, 2025

AI-Generated Code: A Double-Edged Sword for Developers

By Natalie Tischler

If you think AI-generated code is saving time and boosting productivity, you're right. But here's the problem: it's also introducing security vulnerabilities at an alarming rate. Our latest research reveals that **45% of AI-generated code contains security flaws**, turning what should be a productivity breakthrough into a potential security nightmare.

Interested in Learning More?

STARTUP FORTUNE

THURSDAY, APRIL 23, 2026

HOME NEWS AI CRYPTO ENTREPRENEUR INTERVIEWS GUIDES TECH

Lovable's API flaw exposed private project data from the $6.6 billion AI app builder used by Nvidia and Microsoft teams

IA aumenta a superfície de ataque

#7: Segurança

janeiro de 2026, a BlueRock Security analisou 7.000 MCP servers: **36.7% estavam vulneráveis a SSRF** — aquele tipo de vulnerabilidade onde um atacante faz o server fazer requisição pra recursos internos que não deveriam estar expostos. a prova de conceito deles num MCP server da Microsoft conseguiu extrair chaves IAM da AWS direto do metadata endpoint de uma instância EC2. um server mal configurado virando porta de entrada pra cloud inteira.

fevereiro de 2026, a Check Point Research divulgou a CVE-2025-59536: vulnerabilidade crítica no próprio Claude Code (sim, o cliente, não um server aleatório). qualquer `.claude/settings.json` malicioso num repositório clonado conseguia execução remota de código *antes do dialog de confirmação aparecer na tela*. você abria o projeto. o código rodava. fim.

a Trend Micro encontrou 492 MCP servers na internet pública com zero autenticação e zero criptografia. nem HTTPS, camarada.

e a Qualys batizou o fenômeno: "MCP Shadow IT". time de marketing ou de dados sobe um MCP server pra conectar uma ferramenta nova no agente, sem aprovação, sem visibilidade, sem auditoria. igualzinho o SaaS Shadow IT que dominou os anos 2010, só que agora com acesso direto a dados sensíveis via LLM.

MCPs são o novo NPM, com suas vantagens e desvantagens, incluindo segurança!

https://vinineves.com/posts/mcp-ta-onde-o-npm-tava-em-2014/

#7: Segurança

- **Redobre sua atenção.** Não é raro ver AI fazendo uso de práticas inseguras de código.
  - SQL injection, cross-site scripting (XSS), e hard-coded credentials no código, dependências halucinadas, defaults inseguros.
  - Ainda mais se código foi produzido por um não-tech
- **Use ferramentas para identificar vulnerabilidades.** É ainda mais importante continuar usando todas suas ferramentas de SAST, pentesters, etc.

#8: UX e UI

- DHH em um podcast recente:
  - #1 - Beauty is truth, both in technology and in general
  - #2 - Beautiful things bring happiness, whilst horrid things do the opposite
  - #3 - Humans would be happier if aesthetics guided how we build things

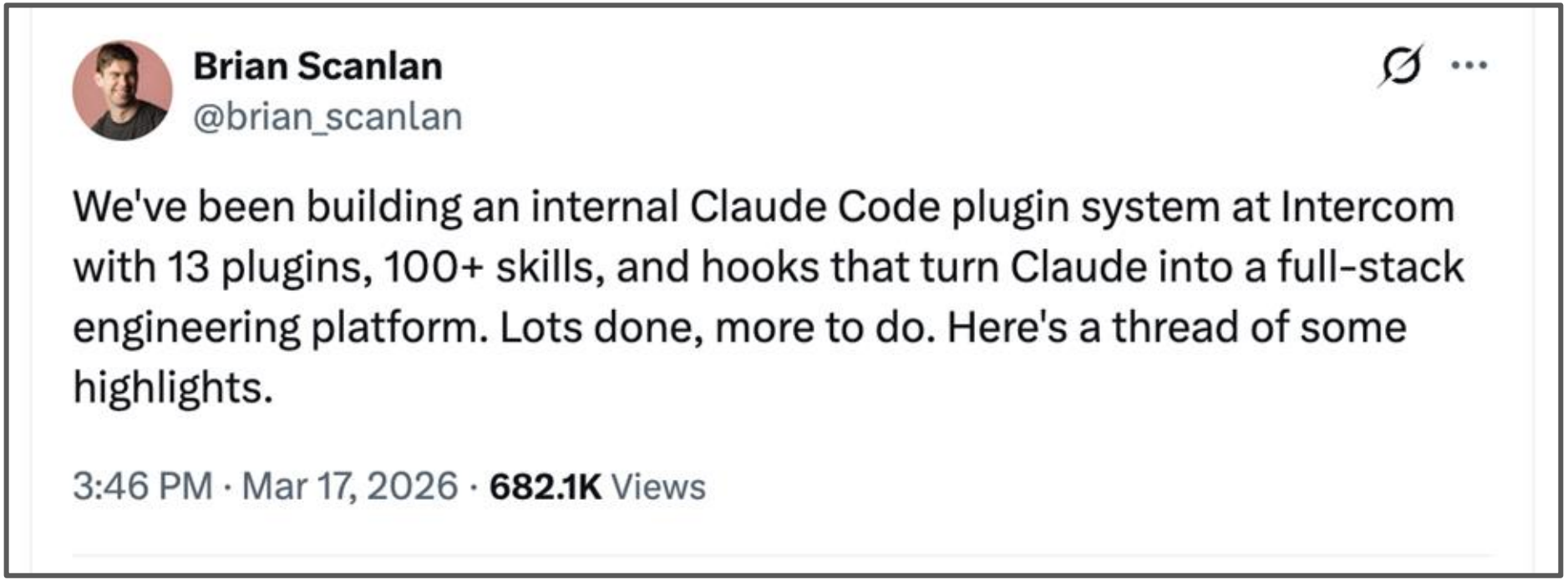
https://www.linkedin.com/posts/why-design-ux-and-aesthetics-matter-3-ugcPost-7452367731989012480-Ct5U

#8: UX e UI

- **UX vai fazer ainda mais diferença.** Escrever código agora é fácil, o difícil é ter bom gosto.
- **Treine seus engenheiros em UX.** A razão UX / dev sempre foi pequena, e agora produziremos ainda mais. Os devs precisam entender de UX para que tudo isso escale.
    - Como escalar "bom gosto"?

O futuro da engenharia de software agêntica

O céu é o limite!



Brian Scanlan
@brian_scanlan

We've been building an internal Claude Code plugin system at Intercom with 13 plugins, 100+ skills, and hooks that turn Claude into a full-stack engineering platform. Lots done, more to do. Here's a thread of some highlights.

3:46 PM · Mar 17, 2026 · **682.1K** Views

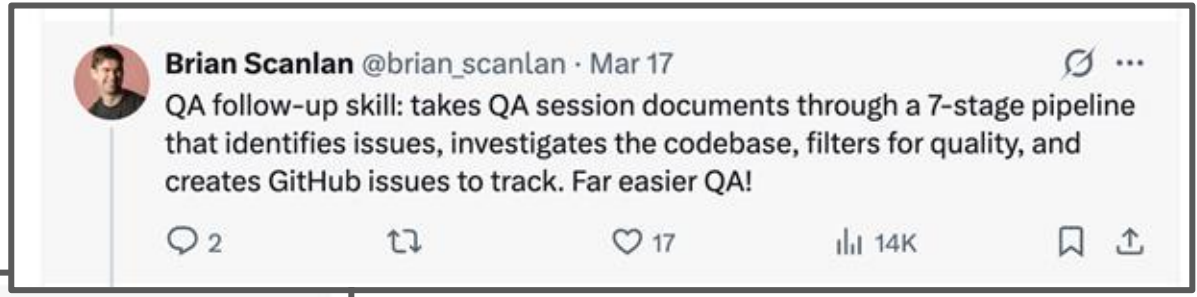
https://x.com/brian_scanlan/status/2033978300003987527?s=46

O céu é o limite!

Agentes saindo do "repositório local" e buscando informações / sinais do seu sistema em produção

https://x.com/brian_scanlan/status/2033978300003987527?s=46

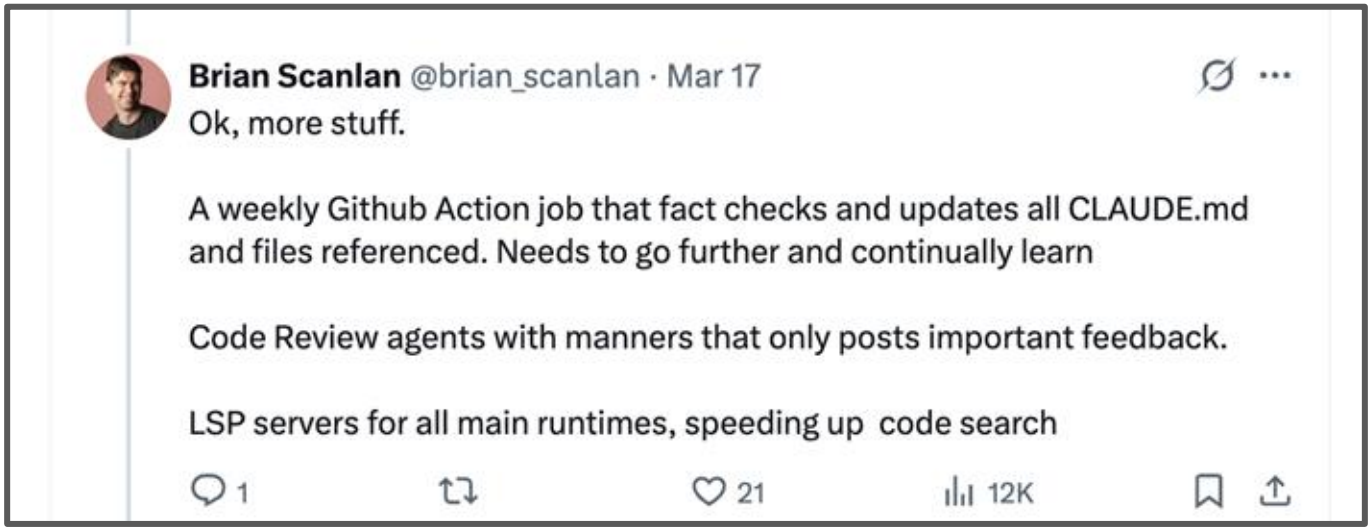
O céu é o limite!



Agentes / skills complexas e sistemáticas que resolvem problemas que sempre foram difíceis e manuais

https://x.com/brian_scanlan/status/2033978300003987527?s=46

O céu é o limite!



Agentes que ficam melhores todo dia / semana, de maneira autônoma

https://x.com/brian_scanlan/status/2033978300003987527?s=46

Níveis de evolução

O Espectro da Evolução: 8 Níveis da Maturidade de IA para Desenvolvedores

A Era do IDE (Níveis 1 a 3)

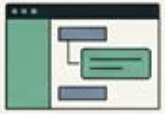
A IA atua como um plugin dentro do ambiente de desenvolvimento tradicional, focada em produtividade individual e tarefas isoladas.

L1: Fluxo Tradicional (Sem IA)

O fluxo de trabalho de engenharia padrão, sem o uso de ferramentas de IA.

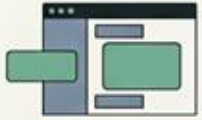
L2: Agente no IDE com Permissões

O desenvolvedor aprova manualmente cada mudança de arquivo; controle total do humano.



L3: Modo YOLO (Confiança em Alta)

O agente opera livremente no IDE enquanto a confiança do desenvolvedor aumenta.



A Ascensão Agent-First (Níveis 4 a 6)

A conversa com a IA torna-se a interface primária. O desenvolvedor deixa de revisar cada linha e passa a guiar a intenção (Vibe Coding).

L4: Liderança pela Conversa

O foco muda da revisão de diffs para o direcionamento estratégico do agente.

L5: Agent-First, IDE depois

O trabalho ocorre na interface do agente; o IDE torna-se apenas um visualizador.

L6: Multiplexação de Agentes

Uso simultâneo de múltiplos agentes em diferentes fluxos para acelerar a entrega.

Orquestração e Autonomia (Níveis 7 a 8)

Gestão de frota. A complexidade exige que o desenvolvedor construa sistemas para gerenciar a própria IA.

L7: Gestão Manual de Multiagentes

Mais de 10 agentes ativos; início da complexidade de contexto e coordenação.

L8: Construção do Próprio Orquestrador

Criação de camadas de coordenação programática para gerenciar e rotear agentes automaticamente.

Resumo de Impacto

Nível	Papel do Desenvolvedor	Foco Principal
L1 - L3	Implementador	Escrita e Sintaxe
L4 - L6	Guia / Revisor	Intenção e Design
L7 - L8	Orquestrador	Arquitetura e Fluxo

alura · FIAP
SKILLS&GO

Qualidade de software no mundo IA

Maurício Aniche

CTO @ Alura, ex-{Uber, Adyen, TU Delft, Locaweb}

Alura | FIAP | PM3 | StartSe